



计算机科学

Computer Science

ISSN 1002-137X, CN 50-1075/TP

《计算机科学》网络首发论文

题目: CausalVulGNN: 基于因果推断与图神经网络的软件漏洞解释方法
作者: 张鑫, 陈文
网络首发日期: 2025-11-20
引用格式: 张鑫, 陈文. CausalVulGNN: 基于因果推断与图神经网络的软件漏洞解释方法[J/OL]. 计算机科学. <https://link.cnki.net/urlid/50.1075.tp.20251119.1735.007>



网络首发: 在编辑部工作流程中, 稿件从录用到出版要经历录用定稿、排版定稿、整期汇编定稿等阶段。录用定稿指内容已经确定, 且通过同行评议、主编终审同意刊用的稿件。排版定稿指录用定稿按照期刊特定版式(包括网络呈现版式)排版后的稿件, 可暂不确定出版年、卷、期和页码。整期汇编定稿指出版年、卷、期、页码均已确定的印刷或数字出版的整期汇编稿件。录用定稿网络首发稿件内容必须符合《出版管理条例》和《期刊出版管理规定》的有关规定; 学术研究成果具有创新性、科学性和先进性, 符合编辑部对刊文的录用要求, 不存在学术不端行为及其他侵权行为; 稿件内容应基本符合国家有关书刊编辑、出版的技术标准, 正确使用和统一规范语言文字、符号、数字、外文字母、法定计量单位及地图标注等。为确保录用定稿网络首发的严肃性, 录用定稿一经发布, 不得修改论文题目、作者、机构名称和学术内容, 只可基于编辑规范进行少量文字的修改。

出版确认: 纸质期刊编辑部通过与《中国学术期刊(光盘版)》电子杂志社有限公司签约, 在《中国学术期刊(网络版)》出版传播平台上创办与纸质期刊内容一致的网络版, 以单篇或整期出版形式, 在印刷出版之前刊发论文的录用定稿、排版定稿、整期汇编定稿。因为《中国学术期刊(网络版)》是国家新闻出版广电总局批准的网络连续型出版物(ISSN 2096-4188, CN 11-6037/Z), 所以签约期刊的网络版上网络首发论文视为正式出版。

CausalVulGNN：基于因果推断与图神经网络的软件漏洞解释方法

张鑫 陈文

四川大学网络空间安全学院 成都 610207
2023226240003@stu.scu.edu.cn

摘要 漏洞解释是软件漏洞挖掘中的关键任务，旨在揭示导致漏洞形成的根本原因，提升软件漏洞的修复质量。现有的漏洞解释方法主要包括基于事实推理和反事实推理两类。其中，事实推理侧重于识别与模型预测结果高度相关的代码子图；反事实推理则通过施加最小扰动改变模型输出，以发现导致漏洞形成的相关代码子图。由于传统方法缺乏显式的代码与漏洞因果关系建模，难以有效区分仅与漏洞成因存在统计相关的伪变量（伪相关子图）与真正导致漏洞形成的因果变量（因果子图）。因此漏洞解释任务容易受到伪相关子图的干扰而产生错误的结论。针对上述问题，提出一种基于代码图神经网络 GNN 的漏洞因果解释方法 CausalVulGNN。该方法首先通过 GNN 多层聚合代码图领域内节点和边信息，在图神经网络的高层学习代码抽象结构-语义特征。随后引入基于样本重加权机制与 Hilbert-Schmidt 独立准则（HSIC）的因果结构模型（Causal Structure Model, CSM），通过建模 GNN 高层表示之间的因果关系以识别代码形成漏洞的因果子图。不同于传统相关性分析方法，CSM 通过模拟高层变量的扰动，量化高层变量对模型预测结果的直接影响，从而识别与漏洞具有因果关系的子图，避免将仅与模型预测结果存在统计相关的伪相关子图错误地纳入漏洞解释过程，从而提升解释结果的正确性。实验结果表明，CausalVulGNN 在多个检测模型与解释器上均取得显著性能提升，验证了其在因果推断视角下的解释能力。在真实大规模漏洞数据集 Big-Vul 上进行了系统性实验，涵盖 GCN、GGNN、GIN、GraphConv 等常用的检测模型。实验结果显示，集成 CausalVulGNN 方法后，常用解释器（CFExplainer、GNNExplainer 和 GNN-LRP）在 Accuracy、Precision、Recall 和 F1 等指标上均获得显著提升，其中 CFExplainer 在 Accuracy 上平均提高了 19.6%，Recall 提升超过 28%，实验结果证实了 CausalVulGNN 在漏洞分析与解释任务中的有效性。

关键词： 因果表示学习；漏洞解释；图神经网络；伪相关性消除；样本重加权；独立性准则

中图法分类号 TP181;TP309

CausalVulGNN: A Framework for Software Vulnerability Explanation Based on Causal Inference and Graph Neural Networks

Zhang Xin, and Chen Wen

School of Cyber Science and Engineering, Sichuan University, Chengdu, China

Abstract Vulnerability explanation is a critical task in software vulnerability mining, aiming to uncover the root causes of vulnerability formation and improve the quality of vulnerability remediation. Existing approaches for vulnerability explanation can be broadly categorized into *factual reasoning* and *counterfactual reasoning* methods. Factual reasoning focuses on identifying code subgraphs that are highly correlated with the model's prediction, while counterfactual reasoning seeks to discover subgraphs responsible for vulnerabilities by applying minimal perturbations that change the model's output. However, traditional methods lack explicit modeling of the causal relationships between code and vulnerabilities, making it difficult to distinguish *spurious variables* (spurious subgraphs) that are only statistically correlated with the vulnerability from the true *causal variables* (causal subgraphs) that directly lead to vulnerability formation. As a result, vulnerability explanation can be misled by spurious correlations, producing inaccurate conclusions. To address this issue, we propose CausalVulGNN, a causal explanation framework for vulnerability analysis based on Graph Neural Networks (GNNs). Our method first

基金项目：国家重点研发计划项目（020YFB1805405）

This work was supported by National Key Research and Development Program of China (No. 020YFB1805405).

通信作者：陈文（wenchen@scu.edu.cn）

employs multi-layer GNN aggregation to capture structural and semantic features by integrating local node and edge information within the code graph. Then, we introduce a *sample reweighting mechanism* and a *Causal Structure Model (CSM)* based on the Hilbert-Schmidt Independence Criterion (HSIC) to model causal relationships among high-level GNN representations. Unlike traditional correlation-based approaches, CSM simulates perturbations to high-level variables and quantifies their direct impact on model predictions, enabling the identification of subgraphs that are truly causal to vulnerabilities. This helps eliminate spurious subgraphs that are only statistically correlated with the output, thereby improving the fidelity of explanations. Experimental results demonstrate that CausalVulGNN achieves significant performance improvements across multiple vulnerability detection models and explanation methods, validating its effectiveness from a causal inference perspective. We conduct systematic experiments on the real-world large-scale vulnerability dataset Big-Vul, covering popular detection models such as GCN, GGNN, GIN, and GraphConv. The results show that integrating CausalVulGNN leads to substantial improvements in standard explanation metrics (Accuracy, Precision, Recall, and F1-score) for widely used explainers including CFExplainer, GNNExplainer, and GNN-LRP. In particular, CFExplainer achieves an average 19.6% increase in Accuracy and over 28% improvement in Recall, confirming the effectiveness of CausalVulGNN for vulnerability explanation and analysis.

Keywords Causal representation learning, Vulnerability explanation, Graph neural networks, Spurious correlation elimination, Sample reweighting, Independence criterion

1. 引言

漏洞解释是网络安全和软件工程领域中重要的质量保证方法，旨在通过识别和分析导致漏洞形成的根本原因，为漏洞修复和系统防护提供有效支持。近年来，随着图数据结构在漏洞建模中的广泛应用，图神经网络（Graph Neural Networks, GNNs）逐渐成为漏洞检测与解释的主流技术^[1,2,3]。

现有的漏洞解释方法，包括基于事实推理（factual reasoning）^[4,5,6]和反事实推理（counterfactual reasoning）^[7]，主要通过相关性分析等手段推断导致漏洞形成的原因。其中，事实推理方法旨在识别与模型预测结果强相关的子图或特征，作为决策的事实证据；反事实推理方法则通过施加最小化扰动改变模型预测结果，从而发现与漏洞形成原因相关的代码子图。然而传统的漏洞解释方法由于缺乏因果关系的建模分析，无法有效区分漏洞的“真正因果变量（因果图）”与“仅与漏洞具有统计相关性的伪变量（伪相关子图）”^[21,22]。

在漏洞解释任务中，伪相关子图^[8]是指仅在训练集中与漏洞标签具有统计相关性，却不是产生漏洞的直接原因的代码结构。这类结构往往因数据分布偏差（如某类API频繁出现在漏洞样本中）或噪声积累（如代码风格、编码习惯）而被错误地学习为“漏洞决策依据”，从而带来以下局限性：

1. 误导性解释：伪相关子图可能干扰对导致漏洞形成根本原因的识别，导致无法有效指导漏洞修复。

2. 缺乏因果关系建模：传统方法未对漏洞的形成原因进行因果建模，难以识别真正导致漏洞形成的代码结构。

事实上，漏洞的形成往往是特定的代码结构或执行逻辑的直接结果，而非由伪相关性导致。传统基于统计相关性分析的漏洞解释方法难以揭示漏洞与导致漏洞的关键代码片段（Sink段）之间的因果关系。

针对上述问题，本文提出了一种基于因果推断^[9]的通用漏洞解释方法。该方法通过因果推断技术消除伪相关性的影响，聚焦于挖掘导致漏洞形成的根本原因。具体而言，本文的方法包括以下关键方面：1). 代码模式高层表示提取：结合可微分代码模式池化层构建神经网络编码器，从程序依赖图中提取代码模式高层表示，通过有序拼接实现语义对齐，为代码子图因果建模漏洞原因分析提供基础。2). 引入因果结构模型：基于样本重加权机制和Hilbert-Schmidt独立准则（HSIC）^[12]，动态调整代码模式高层表示权重，减少伪相关子图的干扰。

本文的主要贡献如下：

1. 因果推断方法的引入：创新性地将因果推断方法引入漏洞解释领域，提高解释结果的因果可信度。

2. Hilbert-Schmidt独立准则的引入：通过动态调整代码子图权重，减少伪相关子图干扰，提升模型的稳定性与解释性。

3. 实验验证有效性：在广泛使用的漏洞数据集 Big-Vul^[10]和多种主流解释器（如CFExplainer^[7]、GNNExplainer^[11]等）进行了广泛的实验评估。实验结果表明，本文方法不仅在解释准确性和泛化性上显著优于现有方法，还能够作为通用的插件增强现有解释器的性能。

总之，本文为漏洞解释领域提供了创新性的解决方案，通过因果推断技术揭示了漏洞形成的

根本原因, 为漏洞修复和系统安全提供了更强有力的支持。

2. 相关工作

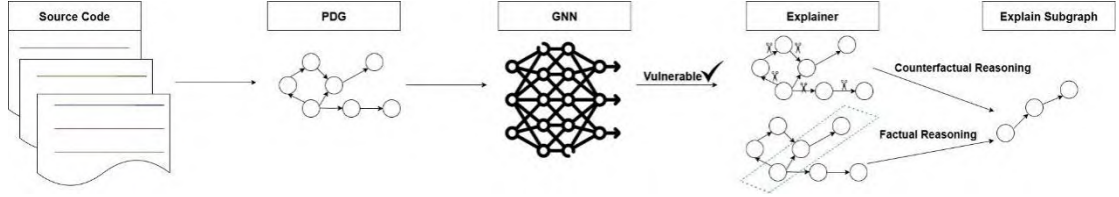


图1 基于图神经网络的漏洞解释流程图

Fig.1 The overall workflow of GNN-based vulnerability explanation.

基于图神经网络 (GNN) 的漏洞解释旨在识别代码中的关键结构, 并提供可解释的漏洞检测结果[4,5,6,7]。基于GNN的漏洞解释流程如图1所示: 首先, 基于源代码的执行顺序和变量的数据依赖关系构建程序依赖图 (PDG), 然后通过事实推理或者反事实推理的解释器来解释图神经网络 (GNN) 的决策过程, 最终生成漏洞解释子图, 即导致漏洞形成的根本原因。

2.1.1 事实推理

事实推理 (Factual Reasoning, FR) 的目标是从代码图 (例如PDG图) 中寻找一个包含足够信息的关键代码子图, 使得仅使用该子图进行漏洞预测时, 图神经网络 (GNN) 模型仍能给出与使用完整代码图相同的预测结果[13]。

FR的定义如下, 给定一个输入代码图 $G_k = \{V_k, E_k\}$ 及训练好的GNN模型预测的标签 $\hat{y}_k$, 希望通过掩码矩阵筛选[14]等方法找到子图 G'_k , 使得:

$$\arg \max_{l \in L} P(l|A_k \odot M_k, X_k \odot F_k) = \hat{y}_k \quad (1)$$

其中, A_k 是原始图的邻接矩阵, M_k 是边掩码矩阵。 X_k 是原始图的节点特征矩阵, F_k 是特征掩码矩阵。 $\odot$ 表示逐元素相乘, 即仅保留原始图中的部分信息。 $P(l|\cdot)$ 表示GNN计算出的类别预测概率, 其中 $L = \{0,1\}$ 代表漏洞或非漏洞。

事实推理代表性工作包括GNNExplainer[11]和GNN-LRP[37]。GNNExplainer旨在通过最大化原始GNN预测结果与子图分布之间的互信息, 以此寻找最关键的子图作为解释。GNN-LRP旨在通过高阶泰勒展开 (Taylor Decomposition) 将GNN的预测分数分解为图中多个路径的贡献度, 从而识别出对预测结果影响最大的路径, 以此作为模型决策的解释依据。

2.1.2 反事实推理

反事实推理 (Counterfactual Reasoning, CR) 的目标是寻找对模型预测结果起决定性作用的边或特征, 即如果这些关键部分被移除, 模型的预测结果就会翻转[15,16,17]。给定一个输入代码图 $G_k = \{V_k, E_k\}$ 及训练好的GNN模型预测的标签 $\hat{y}_k$, 反事实推理的核心目标可以表示如下:

$$\arg \max_{l \in L} P(l|A_k - A_k \odot M_k, X_k - X_k \odot F_k) \neq \hat{y}_k \quad (2)$$

其中, A_k 代表原始图的邻接矩阵, M_k 和 F_k 分别代表边掩码矩阵、特征掩码矩阵。 X_k 是原始图的节点特征矩阵。 $\odot$ 表示逐元素相乘, 用于选择性地删除特定的边或节点特征。 $P(l|\cdot)$ 表示GNN计算

本节介绍本文所涉及的基础知识, 主要包括基于图神经网络的漏洞解释和因果表示学习这两方面。

2.1 基于图神经网络的漏洞解释

出的类别预测概率, 其中 $L = \{0,1\}$ 代表漏洞或非漏洞。

反事实推理代表性工作包括CFExplainer[7]。CFExplainer旨在通过对代码图施加最小扰动, 从而引起模型预测结果翻转, 以此生成反事实解释。

基于事实推理和反事实推理的漏洞解释方法主要依赖相关性分析, 导致其所识别漏洞相关的代码子图不一定是导致漏洞形成的真正因果子图[21,22]。进一步地, 由于代码数据的复杂结构, GNN在训练时不可避免地学习到虚假相关性[18], 从而限制了现有方法的解释能力。

针对以上问题, 本文引入因果推理 (Causal Inference) 区分因果子图 (真正导致漏洞的因素) 和伪相关子图 (仅存在统计相关性但无因果关系的因素)。

2.2 因果表示学习

因果表示学习通过分析数据中变量之间的因果关系, 从原始特征中提取出抽象的因果变量, 并利用因果变量建立对分布变化具有鲁棒性的因果模型[19]。Schölkopf等人[9]指出, 因果性通过聚焦于数据生成过程中的结构化知识, 有助于解决当前机器学习方法在泛化能力弱以及无法区分相关性与因果性等方面的局限性。传统的因果推断方法大多假设观测数据为独立同分布[20], 并可借助因果图明晰变量之间的因果路径。然而, 在源代码分析中, 代码通常被抽象为复杂的图结构数据, 节点之间存在复杂的语义依赖关系 (如函数调用、数据流依赖、控制依赖等), 无法满足传统因果推断方法中的独立性假设条件, 限制了因果表示学习在代码漏洞分析研究中的应用。

为了将因果推断方法应用于图结构数据, 研究者探索适用于代码图等非结构化数据的因果表示学习方法。具体而言, 在处理诸如代码图数据等非结构化数据时, 因果表示学习通常包括两个主要步骤: (1) 从GNN输入特征中提取出抽象的因果变量; (2) 利用因果知识学习抽象变量的因果结构。基于因果表示学习, 不变风险最小化 (Invariant Risk Minimization, IRM) 框架[23,24,25]采用了正则化方法, 即在不同环境下强制模型学习到不变的表示, 其中表示学习部分从原始特征中提取高层抽象表示, 而正则化则将“因果表示在不同环境下应保持不变”的因果知识嵌入到表示学习过程中。IRM框架能够提升模型的泛化能力, 但其对环境标签的依赖限制了在实际应用中的推广。针对这一问题, 研究者提出从数据中隐式地学习环境标签[26,27,28]。然而, 隐式地学习环境标签主要

是针对表格数据或图像数据，难以适用于图等复杂的数据结构。

3. CausalVulGNN方法

本文提出一种用于漏洞解释的因果表示学习方法CausalVulGNN，整体架构如图2所示。CausalVulGNN旨在从程序依赖图（PDG）中挖掘与漏洞存在关键因果关系的子图结构，增强模型的可解释能力。CausalVulGNN包括两个主要模块：代码图特征学习模块与漏洞因果特征区分模块。代码图特征学习模块输入包含漏洞代码与非漏洞代码的PDGs（PDG1~PDGn），每个PDG代表一个函数或代码片段的依赖关系。图中节点代表语句、操作或表达式，边代表数据依赖或控制依赖。首先，CausalVulGNN通过两层级联的池化操作对原始PDG进行压缩。每一层池化操作将结构相似或语义相近的节点聚合为一个簇，以此提取局部结构中的代码模式。代码模式是指代码中具有代表性的结构-语义组合，反映了程序逻辑片

段的特征和语义含义。图中不同颜色的方块代表不同簇的簇特征。将拼接后的代码模式高层表示（簇特征）作为因果变量输入，用于捕捉代码与漏洞相关的潜在因果结构。

由于训练数据中可能存在与漏洞无关但频繁出现的伪相关子图，因此CausalVulGNN引入基于样本重加权的Hilbert-Schmidt独立性准则（HSIC）^[12]的漏洞因果特征区分模块。该模块接收拼接后的代码模式高层表示并输出每个样本的权重（Sample Weights），用于降低非因果变量对预测的干扰。加权后的样本输入GNN分类器进行预测，计算预测值（Prediction）与标签（Label）之间的预测损失（Prediction Loss），并与样本权重相乘得到最终加权损失（Weighted Loss），用于指导模型聚焦于因果变量。综上所述，

CausalVulGNN通过池化操作压缩代码结构、构建统一的代码模式高层表示，借助HSIC消除数据中的非因果因素，使得模型最终对漏洞的预测更具可解释性。

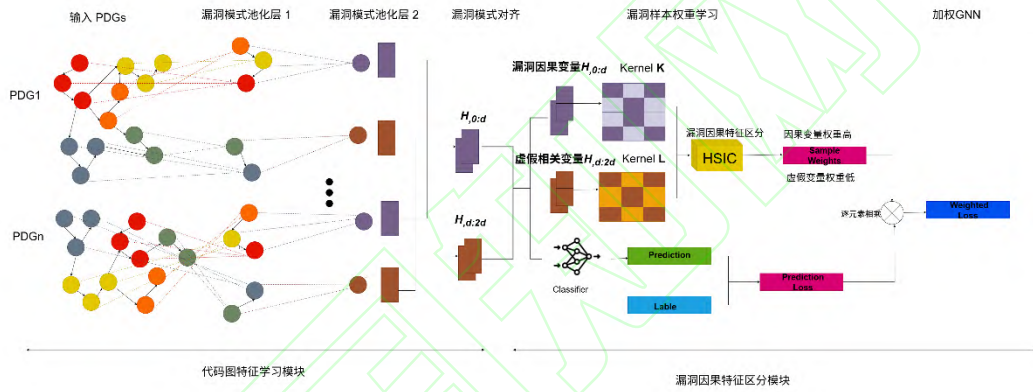


图 2 CausalVulGNN的整体架构

Fig.2 The overall architecture of CausalVulGNN

3.1 代码图特征学习模块

本模块旨在学习一种代码图结构的表示方法，以表征代码图中的关键结构。为此，引入了一种可微分的代码模式池化层，该层将代码图中相互关联的代码片段映射为多个簇（clusters）。随后，根据DiffPool置换不变性^[29]和高层变量语义对齐定理^[32]，通过有序拼接（ordered concatenation）操作，确保所学习的代码模式高层表示在不同代码图之间保持一致的语义对齐，从而提高漏洞解释任务的有效性和稳定性。

漏洞解释任务的目标不仅是识别漏洞及其特征，还需要提供直观的解释以揭示导致漏洞形成的关键代码结构。为此，本文采用DiffPool层^[29]学习代码图中各节点的嵌入表示。DiffPool能够将局部密集连接的子图聚合为簇，从而生成具有层次结构的图表示。这种层次表示有助于解释模型捕捉代码之间以及漏洞内部各组成部分之间的关联性，进而揭示导致漏洞形成的根本原因。

给定输入程序依赖图（PDG）的邻接矩阵 A （下文记作 $A^{(0)}$ ）和描述代码的特征矩阵 F ，使用GNN模块对输入数据进行表示学习，得到节点嵌入矩阵 $F^{(0)}$ ：

$$F^{(0)} = GNN_{embed}^{(0)}(A^{(0)}, F) \quad (3)$$

其中， $GNN_{embed}^{(0)}(\cdot, \cdot)$ 是一个三层GNN模块，具体类型可以是GCN、GGNN、GIN等。特征矩阵 F 通过GraphCodeBERT^[43]的嵌入层提取，用以捕捉源代码片段的上下文语义信息。

随后基于节点嵌入矩阵 $F^{(0)}$ 构建代码模式池化层。首先计算第一层节点的嵌入矩阵：

$$Z^{(1)} = GNN_{embed}^{(1)}(A^{(0)}, F^{(0)}) \quad (4)$$

由于代码图中相互关联的节点往往具有相似的特征和邻接结构，将节点嵌入矩阵 $F^{(0)}$ 与邻接矩阵 $A^{(0)}$ 一同输入到池化GNN模块，以生成第一层的簇分配矩阵：

$$S^{(1)} = \text{softmax}(GNN_{pool}^{(1)}(A^{(0)}, F^{(0)})) \quad (5)$$

其中， $S^{(1)} \in R^{n_0 \times n_1}$ ， n_0 表示输入程序依赖图（PDG）中的节点数量， n_1 表示第一层的簇数。 $S_{i,\cdot}^{(1)}$ 表示第 i 个节点在各簇上的分配概率向量， $S_{\cdot,j}^{(1)}$ 表示所有节点被分配到第 j 个簇的概率， $GNN_{pool}^{(1)}(\cdot, \cdot)$ 是一个三层GNN模块，输出维度为该层预设的最大簇数。Softmax按行操作，为每个节点生成对应的簇分配概率。

得到簇分配矩阵 $S^{(1)}$ 后，即可得到每个节点在各个簇上的分配概率。基于簇分配矩阵 $S^{(1)}$ 和学习

到的节点嵌入矩阵 $Z^{(1)}$ 构造新的聚合图, 其中每个节点表示该层学习得到的簇, 边表示簇之间的连接强度。新图的嵌入矩阵通过公式(6)计算:

$$F^{(1)} = S^{(1)T} Z^{(1)} \in R^{n_1 \times d} \quad (6)$$

其中, d 表示嵌入的维度。公式(6)根据簇分配矩阵 $S^{(1)}$ 对节点嵌入 $Z^{(1)}$ 进行加权聚合, 为每个簇(共 n_1 个)生成表示向量。

类似地, 新图的邻接矩阵通过公式(7)计算:

$$A^{(1)} = (S^{(1)})^T A^{(0)} S^{(1)} \in R^{n_1 \times n_1} \quad (7)$$

公式(4)(5)(6)(7)构成一个完整的代码模式池化层, 可表示为:

$$(A^{(1)}, F^{(1)}) = \text{DiffPool}(A^{(0)}, F^{(0)}) \quad (8)$$

对于第 l 层的代码模式池化层, 可以表示为:

$$(A^{(l)}, F^{(l)}) = \text{DiffPool}(A^{(l-1)}, F^{(l-1)}) \quad (9)$$

通过堆叠多个代码模式池化层, 逐步提取代码图的代码模式表示, 最终获得一个包含关键语义和结构信息的代码模式高层表示:

$$F^{(L)} \in R^{n_L \times d} \quad (10)$$

其中, n_L 是最终代码模式高层表示数量, $F_i^{(L)}$ 表示最终聚合图中第 i 个簇(即由原始输入PDG中一组节点构成的子图)对应的代码模式高层表示。

然而, 由于图数据具有非欧几里得的性质, 对于来自两个不同代码图的分别学习到的第 i 个代码模式高层表示 $F_i^{(L)}$, 其语义含义可能并不一致。例如, 代码图1中的 $F_i^{(L)}$ 可能代表某一代码模式(例如, 正常的输入校验模式), 而代码图2中的 $F_i^{(L)}$ 可能对应于另一种代码模式(例如, 导致漏洞的重复释放资源模式)。为使跨图学习到的代码模式高层表示在语义上实现对齐, 根据DiffPool置换不变性^[29]和高层变量语义对齐定理^[32], 按每个图中代码模式高层表示矩阵 $F^{(L)}$ 的行索引顺序将代码模式高层表示进行拼接:

$$h = \text{concat}(F_1^{(L)}, F_2^{(L)}, \dots, F_{n_L}^{(L)}) \quad (11)$$

其中 $h \in R^{n_L \times d}$, $\text{concat}(\cdot)$ 表示沿行轴的拼接操作。然后, 将一个mini-batch中 m 个代码图的代码模式高层表示矩阵分别拼接后进行堆叠, 得到最终代码模式高层表示矩阵:

$$H = \text{stack}(h_1, h_2, \dots, h_m) \quad (12)$$

其中 h_i 表示第 i 个代码图拼接后的代码模式高层表示矩阵, $\text{stack}(\cdot)$ 表示沿行的堆叠操作。

3.2 漏洞因果特征区分模块

代码图特征学习模块已经提取了所有代码图的代码模式高层表示, 然而高层表示中往往存在虚假相关性, 影响漏洞解释任务的性能。针对这一问题, 首先利用因果推论分析GNN在漏洞解释中的局限性, 然后提出基于样本重加权的Hilbert-Schmidt独立性准则漏洞因果特征区分模块。

基于因果推论分析GNN漏洞解释任务旨在借助GNN学习得到的因果特征变量, 分析模型预测的决策依据, 进而判断哪些代码片段在预测过程中起到了决定作用, 以实现漏洞行为的可解释性建模。然而, 在利用图神经网络(Graph Neural Network)进行训练时, 模型可能会同时学习到既包含因果关系的变量, 又包含与任务无关的具有虚假相关性的变量:

漏洞因果变量(Causal Variables)直接影响漏洞标签, 即真正导致漏洞形成的代码结构。例如,

不安全的指针操作、输入未经过检查、缓冲区溢出等存在漏洞的代码模式。

虚假相关变量(Spurious Variables)不仅与漏洞标签存在统计相关, 还与漏洞因果变量存在结构共现或语义耦合, 从而在代码模式高层表示中形成显著的依赖关系。

在代码图中, 程序结构具有高度的局部语义关联。例如, 真正导致漏洞形成的结构(如越界访问语句)常常嵌套在特定的控制流中(如循环或条件语句), 或伴随着某些函数调用(如字符串处理API)。这使得漏洞因果变量与某些虚假相关变量在图结构上频繁共现, 最终在高层表示空间中引入漏洞因果变量与虚假相关变量之间的统计依赖性。

Hilbert-Schmidt独立性准则能够衡量两个随机变量之间的统计依赖关系。对于两个随机变量 U 和 V 以及核函数 k 和 l , Hilbert-Schmidt独立性准则定义为:

$$\text{HSIC}^{k,l}(U, V) := \|C_{UV}^{k,l}\|_{HS}^2 \quad (13)$$

其中 $C^{k,l}$ 是核 k 和 l 对应的再生核希尔伯特空间中的交叉协方差算子^[12], $\|\cdot\|_{HS}$ 为Hilbert-Schmidt范数。

为了在有限样本条件下估计 $\text{HSIC}^{k,l}(U, V)$, 采用广泛使用的估计器 $\text{HSIC}_0^{k,l}(U, V)$ ^[12], 基于 m 个样本定义为:

$$\text{HSIC}_0^{k,l}(U, V) = (m-1)^{-2} \text{tr}(\text{KPLP}) \quad (14)$$

其中 $K, L \in R^{m \times m}$ 分别为RBF核矩阵, 其元素为 $K_{ij} = k(U_i, U_j)$ 和 $L_{ij} = l(V_i, V_j)$, $P = I - m^{-1} \mathbf{1} \mathbf{1}^T \in R^{m \times m}$ 为中心化矩阵(I 是单位矩阵, $\mathbf{1}$ 是全1列向量)。 P 用于将核矩阵在特征空间中居中, 使其均值为零。

为了消除漏洞因果变量和虚假相关变量之间的依赖性, 通过样本重加权技术调整训练样本在模型学习过程中的权重, 从而生成一个变量间依赖性较低的伪分布^[22]。

因此, 本文引入基于样本重加权的Hilbert-Schmidt独立性准则方法^[32], 旨在消除漏洞因果变量和虚假相关变量之间的依赖性。

对于两个随机变量 U 和 V , 首先使用随机初始权重 w 进行重加权:

$$\hat{U} = (w \cdot \mathbf{1}^T) \odot U \quad (15)$$

$$\hat{V} = (w \cdot \mathbf{1}^T) \odot V \quad (16)$$

其中“ $\odot$ ”表示Hadamard乘积。将 $\hat{U}$ 和 $\hat{V}$ 带入公式(14)中得到加权后的HSIC值:

$$\widehat{\text{HSIC}}_0^{k,l}(U, V, w) = (m-1)^{-2} \text{tr}(\hat{K} \hat{L} P) \quad (17)$$

其中 $\hat{K}, \hat{L} \in R^{m \times m}$ 是加权后的RBF核矩阵, 其元素分别为 $\hat{K}_{ij} = k(\hat{U}_i, \hat{U}_j)$ 和 $\hat{L}_{ij} = l(\hat{V}_i, \hat{V}_j)$ 。

具体而言, 对于漏洞因果变量 $H_{0:d}$ 以及其对应的虚假相关变量集合 $\{H_{d:2d}, \dots, H_{(n_L-1)d:n_L d}\}$, 在多个虚假相关变量之间共享权重 w , 通过公式(18)优化 w :

$$w^* =$$

$$\arg \min_{w \in \Delta_m} \sum_{1 \leq p < n_L} \widehat{\text{HSIC}}_0^{k,l}(H_{0:d}, H_{(p-1)d:n_L d}, w) \quad (18)$$

其中 $\Delta_m = \{w \in R_+^m \mid \sum_{i=1}^m w_i = m\}$ 用于控制每个批次的总体损失几乎不变, 使用 $w = \text{softmax}(w)$ 满足这一约束。因此, 使用最佳权重 w^* 对训练样本进行重加权可以消除漏洞因果变量和虚假相关变量之间的依赖性。

然而, 上述方法只能消除单个漏洞因果变量 $H_{0:d}$ 和虚假相关变量集合 $\{H_{d:2d}, \dots, H_{(nL-1)d:nLd}\}$ 之间的相关性。为了估计所有学习到的代码模式高层表示 $\{H_{0:d}, \dots, H_{(nL-1)d:nLd}\}$ 的因果关系, 将每个代码模式高层表示依次作为漏洞因果变量, 而将其余的代码模式高层表示作为虚假相关变量, 从而依次消除每个漏洞因果变量与相应虚假相关变量之间的依赖性。具体而言, 学习一组样本权重 w 消除所有代码模式高层表示之间的依赖性, 通过公式 (19) 实现:

$$w^* = \arg \min_{w \in \Delta_m} \sum_{1 \leq i < j \leq nL} \widehat{HSIC}_0^{k,l}(H_{(i-1)d:id}, H_{(j-1)d:jd}, w) \quad (19)$$

3.3 基于加权图神经网络的漏洞解释

漏洞因果特征区分模块学习到的样本权重 w 具有去相关性的能力。为了进一步优化权重 w , 本节利用 w 对 GNN 损失函数进行重加权, 并迭代优化样本权重 w 和加权 GNN 模型的参数, 如公式 (20) (21) 所示:

$$f^{(t+1)}, g^{(t+1)} = \arg \min_{f,g} \sum_{i=1}^m w_i^{(t)} L(g(f(A_i, F_i)), y_i) \quad (20)$$

$$w^{(t+1)} = \arg \min_{w^{(t+1)} \in \Delta_m} \sum_{1 \leq i < j \leq nL} \widehat{HSIC}_0^{k,l}(H_{(i-1)d:id}^{(t+1)}, H_{(j-1)d:jd}^{(t+1)}, w^{(t+1)}) \quad (21)$$

其中, $f(\cdot)$ 是代码图特征学习模块, 输出是最终代码模式高层表示矩阵 H ; t 表示迭代次数; $g(\cdot)$ 是线性预测层; $L(\cdot, \cdot)$ 是二元交叉熵损失函数。当固定 GNN 模型参数时, 通过优化目标函数 (21) 来更新样本权重。在获得批次中每个代码样本的权重后, 利用 Adam 优化器^[44] 优化加权 GNN 的目标函数 (20)。在每个 mini-batch 初始阶段, 设定 $w^{(0)} = (1, 1, \dots)^T$ 。在训练过程中, 使用公式 (20) 和 (21) 迭代优化样本权重和模型参数。在推理阶段, 解释器基于训练期间学到的因果特征表示直接执行漏洞解释, 无需再进行样本权重的调整。模型的详细步骤如算法 1 所示。

算法 1: CausalVulGNN 算法流程

输入: 包含漏洞代码与非漏洞代码的 PDG 图

$$G = \{(G_1, y_1), \dots, (G_N, y_N)\}$$

训练轮数: *Epoch*

去相关性轮数: *DecorEpoch*

输出: 加权 GNN 模型

While $t < Epoch$ do

for 1 to BatchNumber do

#[代码图特征学习模块]

#通过分层聚合得到代码模式高层表示矩阵 H , 对应公式 (3) - (12)

$$H = GNN_{Encoder}(G)$$

#[漏洞因果特征区分模块]

for 1 to *DecorEpoch* do

#根据公式 (21) 优化样本权重 w

$$w = OptimizeWeights(H, y);$$

end

#[联合优化]

#根据公式 (20) 使用样本权重更新模型参数

$$Loss = WeightedLoss(f(H), y, w)$$

Backprop(Loss)

end

end

CausalVulGNN 仍在代码数据集 G 上训练, 但是每个代码模式高层表示的权重 w_i 不再相同。该权重调整了每个代码模式高层表示在小批次损失中的贡献, 从而使 GNN 参数能够在去相关化后的代码模式高层表示上进行学习, 更准确地建模代码模式与导致漏洞形成原因之间的真实关联。

本文的方法 CausalVulGNN 旨在通过漏洞因果表示学习方法提高漏洞解释的可靠性。在代码图特征学习模块部分, 利用代码模式池化层来提取代码模式高层表示。在漏洞因果特征区分模块部分, 利用基于加权 Hilbert-Schmidt 独立性准则消除了无意义的虚假相关变量, 使得模型学习到了真正导致漏洞形成的因果变量。这一点对漏洞解释任务至关重要, 并在实验中得到了验证。

4. 实验

本节首先介绍实验所用的数据集、对比实验中选择的解释器以及具体实现细节, 随后, 引入评估指标, 用于定量评估本文所提出的 CausalVulGNN 方法的解释效果。

4.1 数据集

本文采用广泛使用的漏洞检测数据集 Big-Vul^[10], 与现有研究^[3,33,6]保持一致。该数据集构建自公共 CVE 漏洞数据库^[34], 涵盖 2002 年至 2019 年间, 从 348 个开源 C/C++ GitHub 项目中提取的大量源代码漏洞信息。Big-Vul 总共包含 188,636 个函数样本, 其中包含 10,900 个真实漏洞函数, 涵盖 91 种不同类型的安全漏洞。其中, 前十类漏洞 (如 CWE-119、CWE-20、CWE-125、CWE-200、CWE-264 等) 占比约 62.8%, 涵盖了内存管理、输入验证、信息暴露、权限管理等软件系统中最常见的漏洞类型。同时, Big-Vul 中包含大量样本数量较少但类型丰富的漏洞。虽然样本规模有限, 但提升了数据集的多样性和全面性, 有助于验证模型在不同漏洞场景下的稳健性。

与 Devign^[35] 和 Reveal^[5] 等仅提供函数级标签的数据集不同, Big-Vul 提供了基于原始 git 提交记录的语句级漏洞修复信息。为提升数据质量, 本文遵循 Hin 等人^[3] 的清洗流程: 删除注释、忽略纯格

式调整(如缩进、空格)、过滤掉无法解析或被截断的代码段。同时, 对非漏洞样本进行随机欠采样, 以构建正负样本平衡的数据集。因此, BigVul中的修复信息能够更好反映漏洞形成原因, 并为解释质量的评估提供可量化的真实标签。本文将这些语句级修复变更作为构建“解释真值”的依据, 用于客观评估解释器是否准确定位了导致漏洞形成的代码片段。

在图结构数据构建方面, 本文借助代码分析工具 Joern^[36]将每个函数转化为程序依赖图(Program Dependency Graph, PDG), 该图结构是当前图神经网络漏洞检测研究中广泛采用的标准表示^[3,5,33]。在PDG中, 节点表示语句, 边表示控制流或数据流之间的依赖关系。最后, 将构建好的数据随机划分为训练集、验证集和测试集, 比例为8:1:1。在后续的解释生成过程中, 仅针对模型预测结果为漏洞的样本进行解释生成与评估。

4.2 解释器

为了进行对比分析, 本文选取具有代表性的、基于事实推理和反事实推理的GNN解释器作为解释器:

CFExplainer^[7]: 基于反事实推理的GNN解释器。该解释器旨在通过对代码图施加最小扰动, 从而引起模型预测结果翻转, 以此生成反事实解释。

GNNExplainer^[11]: 基于事实推理的GNN解释器。该解释器旨在通过最大化原始GNN预测结果与子图分布之间的互信息, 以此寻找最关键的子图作为解释。

GNN-LRP^[37]: 基于事实推理的GNN解释器。该解释器旨在通过高阶泰勒展开(Taylor Decomposition)将GNN的预测分数分解为图中多个路径的贡献度, 从而识别出对预测结果影响最大的路径, 以此作为模型决策的解释依据。

关于这些基线解释器的超参数设置, 本文采用先前研究中公开的实现参数^[5,38]。

4.3 实现细节

本文实现了四种基于不同GNN核心的漏洞检测模型(即GCN^[39]、GGNN^[40]、GIN^[41]和GraphConv^[42])。每个基础检测模型均采用两层GNN架构, 隐藏层维度为256, 并通过图平均池化(graph mean pooling)生成图级表示。随后, 图级表示被输入到一个两层的MLP分类器中用于漏洞检测。在模型中, 使用GraphCodeBERT的token embedding层^[43]来初始化代码图的节点特征。除最后一层外, 模型在每一层之后都使用ReLU激活函数引入非线性。

为了验证本文提出方法的有效性, 分别在未使用CausalVulGNN方法的基础模型(记为GNN)和集成了CausalVulGNN方法的模型(记为CausalVulGNN)上进行对比实验。在集成版本中, 本文在两层GNN之间插入一层代码模式池化层, 并分别采用GCN、GGNN、GIN和GraphConv作为CausalVulGNN的底层模块, 以提取代码图的高层语义结构。

所有模型均采用二元交叉熵损失函数, 使用Adam优化器^[44]训练50轮, 学习率为0.005, 批量大

实验结果如表1, 2, 3所示, 集成了CausalVulGNN方法的模型在解释性能上整体优于

小为64。最终模型在验证集上表现最优的轮次被用于测试阶段的评估。

解释器旨在揭示对漏洞检测起决定性作用的关键因素, 因此解释非漏洞代码片段并无实际价值; 同时, 对于被误判为漏洞的样本进行解释也会引入偏差, 影响评估的公正性。因此, 本文将解释范围限定在被准确识别为漏洞的代码样本上, 以确保解释的有效性与评估的一致性。

4.4 漏洞解释评估指标

在评估漏洞解释时, 本文遵循漏洞检测研究中的通用方法^[3,5,33], 将提交记录中被删除或修改的语句(标记为“-”)视为漏洞相关的“真值”标签。具体而言, 从每个漏洞函数的原始代码中提取所有语句, 并构建一个二值标签向量 $S = [s_1, s_2, \dots, s_r]$, 其中 $s_i=1$ 表示第 i 条语句在后续的修复版本中被移除或修改, $s_i=0$ 则表示该语句未发生变动。此向量作为评估中用于衡量解释结果准确性的参考标准。

相应地, 为每个生成的解释子图构建一个二值解释向量 $\Delta = [\delta_0, \delta_1, \dots, \delta_r]$, 其中 δ_i 为非零值表示第 i 条语句被包含在该解释子图中。将该解释向量 Δ 与预先构建的真值向量 S 进行比较, 从而实现解释结果的量化评估。

对于一个漏洞代码样本集合 $\{C_1, C_2, \dots, C_m\}$, 如果解释中成功包含了标注为“修复相关”的语句(即至少包含一个被删除或修改的语句), 则该解释被视为正确解释。统计所有样本中解释正确的比例, 计算解释的准确率(Accuracy), 计算方式如下:

$$Accuracy = \frac{1}{M} \sum_{k=1}^M \mathbb{I}[\Delta^{(k)} \cap S^{(k)} \neq \emptyset] \quad (22)$$

其中, M 表示总的样本数; $\Delta^{(k)}$ 表示第 k 个样本的解释子图中包含的语句索引集合; $S^{(k)}$ 表示第 k 个样本中标注为“真值”的语句集合; $\mathbb{I}[\cdot]$ 是指示函数, 当括号内条件为真时为1, 反之为0。此外, 计算每个样本的精确率(Precision)和召回率(Recall), 计算方式如下:

$$Precision = \frac{\sum_{i=1}^r s_i \delta_i}{\sum_{i=1}^r \delta_i} \quad (23)$$

$$Recall = \frac{\sum_{i=1}^r s_i \delta_i}{\sum_{i=1}^r s_i} \quad (24)$$

其中, Precision表示解释中与漏洞相关语句所占比例, 用于衡量解释内容的精确性; Recall表示所有真实漏洞语句被正确解释的比例, 衡量解释的覆盖性。综合这两个指标, 进一步计算F1值作为整体评估表现, 计算方式如下:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \quad (25)$$

最后, 对所有代码样本计算平均Precision、Recall和F1分数, 用于总体评估解释器性能。

4.5 实验结果

为全面评估各解释器在漏洞解释任务中的表现, 本文在四种GNN检测模型(GCN、GGNN、GIN、GraphConv)及其集成CausalVulGNN方法的模型(CausalVulGCN、CausalVulGGNN、CausalVulGIN、CausalVulGraphConv)上分别结合三种解释器(CFExplainer、GNNExplainer、GNN-LRP)进行实验, 并采用准确率(Accuracy)、精确率(Precision)、召回率(Recall)与F1分数四项指标进行定量评估。

未集成版本。无论是采用何种解释器, CausalVulGNN模型在所有四项评估指标上均表现

出更强的解释能力。以CFExplainer为例，在Accuracy上平均提高了19.6%，Recall提升超过28%，显示出CausalVulGNN方法在辅助解释器定位漏洞因果图的重要作用。

4.6消融实验

CausalVulGNN方法由代码图特征学习模块和漏洞因果特征区分模块（VCFD）两部分构成，为了研究每个模块的作用，进行消融实验。对于移

除漏洞因果特征区分模块的模型，记为CausalVulGNN-NoVCFD。实验结果如表1，2，3所示。结果显示：CausalVulGNN-NoVCFD性能优于GNN，验证了代码图特征学习模块在漏洞解释任务中是必要且有效的。CausalVulGNN性能优于CausalVulGNN-NoVCFD，验证了漏洞因果特征区分模块能够有效区分并消除虚假相关变量的影响，进一步提升模型的解释能力。

```
CVE-2012-2816
1 void WebPluginProxy::SetWindowlessPumpEvent(HANDLE pump_messages_event) {
2     HANDLE pump_messages_event_for_renderer = NULL;
3     DuplicateHandle(GetCurrentProcess(), pump_messages_event, channel_>renderer_handle(), &pump_messages_event_for_renderer, 0, FALSE, DUPLICATE_SAME_ACCESS);
4     sandbox::BrokerDuplicateHandle(pump_messages_event, channel_>peer_pid(), &pump_messages_event_for_renderer, SYNCHRONIZE | EVENT_MODIFY_STATE, 0);
5     DCHECK(pump_messages_event_for_renderer != NULL);
6     Send(new PluginHostMsg_SetWindowlessPumpEvent{
7         route_id_, pump_messages_event_for_renderer});
8 }
9
10 void WebPluginProxy::CreateCanvasFromHandle(
11     const TransportDIB::Handle& dib_handle,
12     const gfx::Rect& window_rect,
13     scoped_ptr<skia::PlatformCanvas>* canvas_out) {
14     DuplicateHandle(channel_>renderer_handle(), dib_handle, GetCurrentProcess(), &section, STANDARD_RIGHTS_REQUIRED | FILE_MAP_READ | FILE_MAP_WRITE, FALSE, 0);
15     scoped_ptr<skia::PlatformCanvas> canvas(new skia::PlatformCanvas);
16     if (!canvas->initialize(
17         window_rect.Width(), window_rect.Height(), true,
18         section)) {
19         CloseHandle(section);
20         dib_handle->reset();
21         canvas_out->reset();
22     }
23     canvas_out->reset(canvas.release());
24     CloseHandle(section);
25     CloseHandle(dib_handle);
26 }
```

图3 CVE-2012-2816漏洞实例分析

Fig.3 A case study on the CVE-2017-13001 vulnerability

4.7实例分析

为了进一步验证方法的有效性，如图3所示，选取了CVE-2012-2816漏洞实例进行解释性分析。新增的代码行用“+”号标记，删除的代码行用“-”号标记。漏洞产生的原因在于：原始代码在将事件与共享内存句柄跨进程传递时，直接使用 DuplicateHandle 并沿用 DUPLICATE_SAME_ACCESS（第3行、第14行），这会导致句柄权限过大，违背最小权限原则，进而带来潜在的沙箱边界绕过风险与错误的所有权语义；同时，额外复制的 section 句柄依赖 CloseHandle(section)（第23行）进行释放，增加了资源泄漏与生命周期管理不当的概率。为了修复该漏洞，在 SetWindowlessPumpEvent 中，引入 sandbox::BrokerDuplicateHandle，并将目标进程明确为 channel_>peer_pid()，同时将事件的访问权限收敛为 SYNCHRONIZE | EVENT_MODIFY_STATE。改动用受控的 broker 通道替代了直接的跨进程复制，并将权限限制在事件同步与状态修改所必需的最小集合，降低了不必要的特权暴露。在 CreateCanvasFromHandle 中，去除了对共享内存

句柄再次复制为本地 section 的做法，直接使用传入的 dib_handle 完成

PlatformCanvas::initialize，并在使用后显式 CloseHandle(dib_handle)。既避免了多余的句柄复制与宽权限授予，也简化了所有权与清理逻辑，降低了资源泄漏与越权访问的可能性。在实例分析中，比较了集成CausalVulGCN方法的模型与未集成该方法的GCN模型在CFExplainer、GNNExplainer、GNN-LRP三种解释器下的解释效果。结果表明，未集成CausalVulGCN的模型存在明显局限性：其中CFExplainer 仅识别到第14行，GNNExplainer虽然能够定位第3行，但包含若干与漏洞无关的语句，削弱了解释的清晰性；GNN-LRP则未能识别到导致漏洞形成的关键代码行。相比之下，集成CausalVulGCN后，模型的解释效果显著提升：CFExplainer能够准确定位第3行、第14行和第23行，GNNExplainer 识别出第3行和第14行，GNN-LRP也成功识别到第3行。实例分析结果表明，CausalVulGCN能够有效增强不同解释器的解释能力，使其聚焦于导致漏洞形成的关键代码语句，从而提升解释结果的针对性与实用性。

表1 CFExplainer在GNN与CausalVulGNN模型下的漏洞解释性能对比

Table 1 Comparison of CFExplainer's vulnerability explanation performance between GNN and CausalVulGNN

	Accuracy	Precision	Recall	F1
GCN	0.5932	0.1256	0.4289	0.1661
CausalVulGCN-	0.6218	0.1312	0.4527	0.1783

NoVCFD				
CausalVulGCN	0.6750	0.1437	0.5115	0.2019
GGNN	0.5564	0.1259	0.3704	0.1566
CausalVulGGNN- NoVCFD	0.5987	0.1298	0.4127	0.1679
CausalVulGGNN	0.6359	0.1341	0.4504	0.1806
GIN	0.5436	0.1297	0.3900	0.1615
CausalVulGIN- NoVCFD	0.6158	0.1351	0.4689	0.1839
CausalVulGIN	0.6920	0.1456	0.5285	0.2027
GraphConv	0.5632	0.1316	0.3729	0.1609
CausalVulGraphConv- NoVCFD	0.6039	0.1368	0.4387	0.1754
CausalVulGraphConv	0.6920	0.1529	0.5099	0.2099

表2 GNNExplainer在GNN与CausalVulGNN模型下的漏洞解释性能对比

Table 2 Comparison of GNNExplainer's vulnerability explanation performance between GNN and CausalVulGNN

	Accuracy	Precision	Recall	F1
GCN	0.5665	0.1256	0.4077	0.1582
CausalVulGCN- NoVCFD	0.5758	0.1289	0.4215	0.1639
CausalVulGCN	0.6036	0.1427	0.4694	0.1939
GGNN	0.6109	0.1199	0.4088	0.1582
CausalVulGGNN- NoVCFD	0.6198	0.1235	0.4176	0.1635
CausalVulGGNN	0.6357	0.1298	0.4289	0.1740
GIN	0.4826	0.1135	0.3245	0.1385
CausalVulGIN- NoVCFD	0.5524	0.1328	0.4051	0.1768
CausalVulGIN	0.6775	0.1512	0.5329	0.2102
GraphConv	0.5235	0.1255	0.3282	0.1470
CausalVul GraphConv- NoVCFD	0.5679	0.1368	0.4125	0.1694
CausalVul GraphConv	0.6298	0.1505	0.4857	0.2034

表3 GNN-LRP在GNN与CausalVulGNN模型下的漏洞解释性能对比

Table 3 Comparison of GNN-LRP's vulnerability explanation performance between GNN and CausalVulGNN

	Accuracy	Precision	Recall	F1
GCN	0.5862	0.1367	0.4255	0.1730
CausalVulGCN- NoVCFD	0.5897	0.1371	0.4692	0.1849
CausalVulGCN	0.5950	0.1362	0.5145	0.1956
GGNN	0.5097	0.1088	0.3501	0.1398
CausalVulGGNN- NoVCFD	0.5284	0.1175	0.3947	0.1547
CausalVulGGNN	0.5456	0.1262	0.4405	0.1709
GIN	0.4867	0.1181	0.3307	0.1434
CausalVulGIN- NoVCFD	0.5318	0.1292	0.3946	0.1647
CausalVulGIN	0.6014	0.1438	0.5254	0.2031
GraphConv	0.4815	0.1235	0.3209	0.1476
CausalVul GraphConv- NoVCFD	0.5282	0.1318	0.3796	0.1613
CausalVul GraphConv	0.5868	0.1408	0.4959	0.1977

综上所述，实验结果表明：集成CausalVulGNN方法的GNN模型能够提取更具代码语义结构抽象性的代码模式高层表示，从而增强解释器识别漏洞因果图的能力。这充分验证了本文提出的CausalVulGNN方法在漏洞解释任务中的有效性与先进性。

结束语

本文围绕图神经网络（GNN）在漏洞检测任务中的可解释性问题，创新性地提出了一种基于因果推断的通用解释方法——CausalVulGNN，旨在从因果视角揭示导致漏洞形成的根本原因，并有效区分具有因果效应的子图与仅存在统计相关性的伪相关子图，从而提升解释结果的准确性。

通过引入因果结构建模，CausalVulGNN能够有效识别导致漏洞形成的具有真实因果效应的子图结构，从而提供更具准确性和实用性的解释结果。实验在大规模真实漏洞数据集Big-Vul上展开，涵盖GCN、GGNN、GIN、GraphConv等多种主流检测模型。实验结果表明，CausalVulGNN在Accuracy、Precision、Recall以及F1等多项指标上均优于现有主流解释方法，验证了其在准确性和稳定性方面的优势。

此外，该方法通用性好，可作为解释增强插件集成至现有GNN解释器中，为漏洞检测系统提供因果关系建模的可解释机制。在当前因果表示学习与图学习不断融合的背景下，

CausalVulGNN为GNN可解释性研究提供一定的理论与方法补充，同时也为图级因果因子提取、跨项目漏洞归因等后续方向探索提供了一种可行的思路与实现范式。

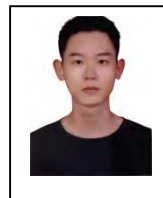
未来工作中，将进一步探索因果结构学习在代码图建模中的自动化实现，尝试在更复杂的现实环境中，如多语言项目、多源异构代码仓库等场景中验证方法的鲁棒性与通用性，并推动其在软件工程中其他关键任务（如补丁推荐、代码审计等）中的广泛应用。

参考文献

- [1] Cheng X, Wang H, Hua J, et al. Deepwukong: Statically detecting software vulnerabilities using deep graph neural network[J]. ACM Transactions on Software Engineering and Methodology, 2021, 30(3): 1-33.
- [2] Guo W, Fang Y, Huang C, et al. HyVulDect: A hybrid semantic vulnerability mining system based on graph neural network[J]. Computers & Security, 2022, 121: 102823.
- [3] Hin D, Kan A, Chen H, et al. Linevd: Statement-level vulnerability detection using graph neural networks[C]//Proceedings of the 19th International Conference on Mining Software Repositories. ACM, 2022: 596-607.

- [4] Ganz T, Häterich M, Warnecke A, et al. Explaining graph neural networks for vulnerability discovery[C]//Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security. ACM, 2021: 145-156.
- [5] Hu Y, Wang S, Li W, et al. Interpreters for GNN-based vulnerability detection: Are we there yet?[C]//Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. ACM, 2023: 1407-1419.
- [6] Li Y, Wang S, Nguyen T N. Vulnerability detection with fine-grained interpretations[C]//Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ACM, 2021: 292-303.
- [7] Chu Z, Wan Y, Li Q, et al. Graph neural networks for vulnerability detection: A counterfactual explanation[C]//Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. ACM, 2024: 389-401.
- [8] Miao S, Liu M, Li P. Interpretable and generalizable graph learning via stochastic attention mechanism[C]//Proceedings of the International Conference on Machine Learning. PMLR, 2022: 15524-15543.
- [9] Schölkopf B, Locatello F, Bauer S, et al. Toward causal representation learning[J]. *Proceedings of the IEEE*, 2021, 109(5): 612-634.
- [10] Fan J, Li Y, Wang S, et al. AC/C++ code vulnerability dataset with code changes and CVE summaries[C]//Proceedings of the 17th International Conference on Mining Software Repositories. ACM, 2020: 508-512.
- [11] Ying Z, Bourgeois D, You J, et al. GNNExplainer: Generating explanations for graph neural networks[J]. *Advances in Neural Information Processing Systems*, 2019, 32: 9240-9251.
- [12] Gretton A, Bousquet O, Smola A, et al. Measuring statistical dependence with Hilbert-Schmidt norms[C]//Algorithmic Learning Theory. Berlin: Springer, 2005: 63-77.
- [13] Narasimhan M, Lazebnik S, Schwing A. Out of the box: Reasoning with graph convolution nets for factual visual question answering[J]. *Advances in Neural Information Processing Systems*, 2018, 31: 1014-1024.
- [14] Luo L, Li Y F, Haffari G, et al. Reasoning on graphs: Faithful and interpretable large language model reasoning[EB/OL]. arXiv:2310.01061, 2023[2025-09-23]. Available: <https://arxiv.org/abs/2310.01061>.
- [15] Abrate C, Bonchi F. Counterfactual graphs for explainable classification of brain networks[C]//Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. ACM, 2021: 2495-2504.
- [16] Li Q, Wang X, Wang Z, et al. Be causal: De-biasing social network confounding in recommendation[J]. *ACM Transactions on Knowledge Discovery from Data*, 2023, 17(1): 1-23.
- [17] Li Q, Wang Z, Liu S, et al. Causal optimal transport for treatment effect estimation[J]. *IEEE Transactions on Neural Networks and Learning Systems*, 2021, 34(8): 4083-4095.
- [18] Ye W, Zheng G, Cao X, et al. Spurious correlations in machine learning: A survey[EB/OL]. arXiv:2402.12715, 2024[2025-09-23]. Available: <https://arxiv.org/abs/2402.12715>.
- [19] Yang M, Liu F, Chen Z, et al. CausalVAE: Disentangled representation learning via neural structural causal models[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. IEEE, 2021: 9593-9602.
- [20] Liao R, Urtasun R, Zemel R. A pac-bayesian approach to generalization bounds for graph neural networks[EB/OL]. arXiv:2012.07690, 2020[2025-09-23]. Available: <https://arxiv.org/abs/2012.07690>.
- [21] Zhang X, Cui P, Xu R, et al. Deep stable learning for out-of-distribution generalization[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. IEEE, 2021: 5372-5382.
- [22] Lake B M, Ullman T D, Tenenbaum J B, et al. Building machines that learn and think like people[J]. *Behavioral and Brain Sciences*, 2017, 40: e253.
- [23] Arjovsky M, Bottou L, Gulrajani I, et al. Invariant risk minimization[EB/OL]. arXiv:1907.02893, 2019[2025-09-23]. Available: <https://arxiv.org/abs/1907.02893>.
- [24] Rosenfeld E, Ravikumar P, Risteski A. The risks of invariant risk minimization[EB/OL]. arXiv:2010.05761, 2020[2025-09-23]. Available: <https://arxiv.org/abs/2010.05761>.
- [25] Kamath P, Tangella A, Sutherland D, et al. Does invariant risk minimization capture invariance?[C]//Proceedings of the International Conference on Artificial Intelligence and Statistics. PMLR, 2021: 4069-4077.
- [26] Qiao F, Zhao L, Peng X. Learning to learn single domain generalization[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. IEEE, 2020: 12556-12565.
- [27] Matsuura T, Harada T. Domain generalization using a mixture of multiple latent domains[C]//Proceedings of the AAAI Conference on Artificial Intelligence. AAAI, 2020, 34(7): 11749-11756.

- [28] Wang H, He Z, Lipton Z C, et al. Learning robust representations by projecting superficial statistics out[EB/OL]. arXiv:1903.06256, 2019[2025-09-23]. Available: <https://arxiv.org/abs/1903.06256>.
- [29] Ying Z, You J, Morris C, et al. Hierarchical graph representation learning with differentiable pooling[J]. Advances in Neural Information Processing Systems, 2018, 31: 4805-4815.
- [30] Hainmueller J. Entropy balancing for causal effects: A multivariate reweighting method to produce balanced samples in observational studies[J]. Political Analysis, 2012, 20(1): 25-46.
- [31] Zubizarreta J R. Stable weights that balance covariates for estimation with incomplete outcome data[J]. Journal of the American Statistical Association, 2015, 110(511): 910-922.
- [32] Fan S, Wang X, Shi C, et al. Generalizing graph neural networks on out-of-distribution graphs[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2023, 46(1): 322-337.
- [33] Fu M, Tantithamthavorn C. LineVul: A transformer-based line-level vulnerability prediction[C]//Proceedings of the 19th International Conference on Mining Software Repositories. ACM, 2022: 608-620.
- [34] Gu M, Feng H, Sun H, et al. Hierarchical attention network for interpretable and fine-grained vulnerability detection[C]//IEEE INFOCOM 2022-IEEE Conference on Computer Communications Workshops. IEEE, 2022: 1-6.
- [35] Zhou Y, Liu S, Siow J, et al. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks[J]. Advances in Neural Information Processing Systems, 2019, 32: 9570-9581.
- [36] Yamaguchi F, Golde N, Arp D, et al. Modeling and discovering vulnerabilities with code property graphs[C]//Proceedings of the IEEE Symposium on Security and Privacy. IEEE, 2014: 590-604.
- [37] Schnake T, Eberle O, Lederer J, et al. Higher-order explanations of graph neural networks via relevant walks[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2021, 44(11): 7581-7596.
- [38] Yuan H, Yu H, Gui S, et al. Explainability in graph neural networks: A taxonomic survey[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2022, 45(5): 5782-5799.
- [39] Kipf T N, Welling M. Semi-supervised classification with graph convolutional networks[EB/OL]. arXiv:1609.02907, 2016[2025-09-23]. Available: <https://arxiv.org/abs/1609.02907>.
- [40] Li Y, Tarlow D, Brockschmidt M, et al. Gated graph sequence neural networks[EB/OL]. arXiv:1511.05493, 2015[2025-09-23]. Available: <https://arxiv.org/abs/1511.05493>.
- [41] Xu K, Hu W, Leskovec J, et al. How powerful are graph neural networks?[EB/OL]. arXiv:1810.00826, 2018[2025-09-23]. Available: <https://arxiv.org/abs/1810.00826>.
- [42] Morris C, Ritzert M, Fey M, et al. Weisfeiler and leman go neural: Higher-order graph neural networks[C]//Proceedings of the AAAI Conference on Artificial Intelligence. AAAI, 2019, 33(1): 4602-4609.
- [43] Guo D, Ren S, Lu S, et al. GraphCodeBERT: Pre-training code representations with data flow[EB/OL]. arXiv:2009.08366, 2020[2025-09-23]. Available: <https://arxiv.org/abs/2009.08366>.
- [44] Kingma D P, Ba J. Adam: A method for stochastic optimization[EB/OL]. arXiv:1412.6980, 2014[2025-09-23]. Available: <https://arxiv.org/abs/1412.6980>.
- 张鑫, 出生于1998年, 硕士, 主要研究方向为机器学习、漏洞挖掘。
- 陈文, 出生于1983年, 博士, 副教授, 博士生导师, CCF 会员, 主要研究方向为网络安全、数据挖掘。



Zhang Xin, born in 1998, postgraduate, is not a member of CCF. His main research interests include machine learning and vulnerability mining.



Chen Wen, born in 1983, PhD, associate professor, PhD supervisor, is a member of CCF. His main research interests include cyber security and data mining.